

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №2
по дисциплине «Методы оптимизации»
Тема: «Метод ломаных»

Выполнила:
Степанова Екатерина Андреевна, К3341
поток 1.2

Санкт-Петербург
2026 г.

Задание

Разработайте программу на языке Python или подобном (C++, Java...), которая будет решать задачу поиска глобального экстремума для заданной функции на заданном отрезке.

На вход программа принимает строку одномерной функции, например $f(x) = x +$

$\sin(3.14159 \cdot x)$ (она заведомо должна являться липшицевой), координаты концов

отрезка (вещественные числа), точность вычисления ϵ (например, 0,01).

На выходе

она выполняет визуализацию графика исходной функции,

вспомогательных функций

(по возможности итоговой ломаной, на которой достигнута требуемая точность),

приближенное значение аргумента и минимального значения функции, число

пробовавшихся итераций, потраченной время.

Программа должна быть либо в Jupiter Notebook или в репозитории.

Должно быть

продемонстрирована работа программы на тестовой функции (любой одномерный

сред двумерной функции): функция Растригина, функция Экли и др., но с наличием

не менее двух локальных минимумов на заданном отрезке.

Результаты расчетов должны отражаться в виде визуализации (функция, ломаная

линия или вспомогательные функции, указание места найденного приближенного

значения аргумента и соответствующей точки минимума), приближенные значения

аргумента и функции, число итераций, затраченное время для расчетов.

Демонстрация должна быть отражена в PDF файле, который вместе ссылкой на файл с кодом, с файл визуализация и(или) файл Jupiter Notebook, размещается в Moodle в ответ на это задание.

Теория

Метод пиявского(ломаных) - метод нахождения глобального минимума(максимума) липшицевой функции.

Функция $W(x)$ называется Липшицевой, если при любых $u, v \in [a, b]$ найдется константа $L > 0$, при которой выполняется:

$$|W(u) - W(v)| \leq L|u - v|.$$

Иными словами при любых $u, v \in [a, b]$, прирост функции между этими точками по модулю будет МЕНЬШЕ, чем прирост самого аргумента по модулю, помноженного на константу.

Основной конструкционный элемент:

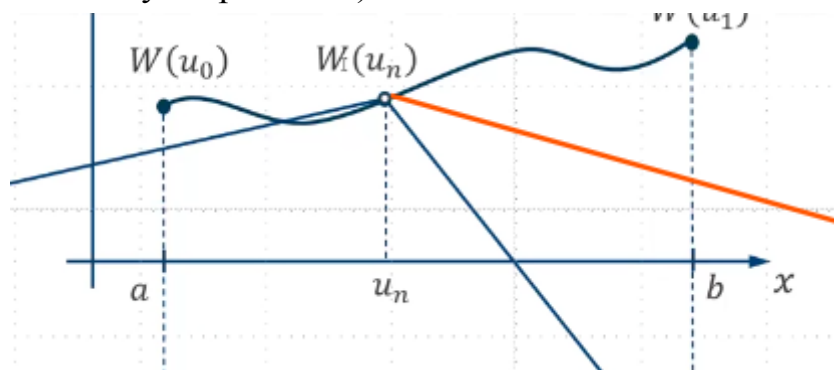
$$g_n(x, u_n) = W(u_n) - L|x - u_n|$$

g - звено ломаной - основной конструкционный элемент, из которого собирается итоговая сложная функция(является одной из многих деталей, которые, пересекаясь, образуют непрерывную прямую)

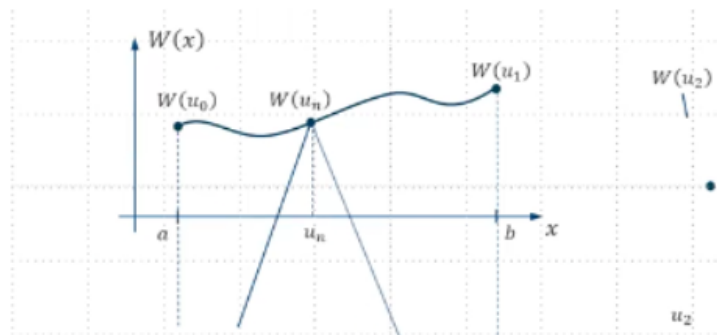
Число L называется *константой Липшица*.

Геометрически константа Липшица определяет наклон основного конструкционного элемента.

Если эту константу недооценить(слишком малая по модулю), то может быть такое, она пробьет нашу функцию, появятся искаженные решения(метод не будет работать)



Если L переоценить, то звено будет под более большим углом, метод будет работать, но дольше, а=появятся риски с вычислительной точностью.



Выполнение

В отчете я предоставляю реализацию только важных, по моему мнению частей: нахождение константы Липшица и метод Пиявского(ломаных).

Метод Пиявского

```
1. def piyavsky_method(W, a, b, eps, L, max_iter = 500):
2.     time_start = time.perf_counter()
3.     #верхние границы
4.     u = [a, b]
5.     Wu = [W(a), W(b)]
6.     n = 1
7.     while n < max_iter:
8.         n+=1
9.         # нижние вершины
10.        lower = []
11.        for i in range(len(u)-1):
12.            u_cross = (Wu[i]-Wu[i+1])/(2*L) + (u[i] +
13.                u[i+1])/2
14.            p_cross = (Wu[i]+Wu[i+1])/2 - L*(u[i+1] -
15.                u[i])/2
16.            lower.append((p_cross, u_cross, i))
17.            p_new, u_new, i = min(lower)
18.            u.insert(i+1, u_new)
19.            W_n = W(u_new)
20.            delta = W_n - p_new
21.            Wu.insert(i+1, W_n)
22.            if delta < eps:
23.                break
24.            k = Wu.index(min(Wu)) # индекс глобального
25.                экстремума
26.        time_end = time.perf_counter()
27.        return {"u_star": u[k], "W_star": Wu[k],
28.            "iterations": n - 1, "delta": delta,
29.            "time": time_end-time_start, "u": u, "Wu":
30.                Wu, "lower": [(x, p) for p, x, _ in lower]}
```

На вход функция принимает:

W - данную функцию

a, b - рассматриваемый отрезок

eps - точность вычисления

L - константа Липшица

max_iter - количество максимальных итераций(для отслеживания)

Алгоритм:

На каждой итерации программа:

1. рассматривает все соседние верхние вершины;
2. вычисляет координаты точек пересечения;
3. вычисляет значения ломаной в этих точках;
4. выбирает минимальную нижнюю вершину;
5. вычисляет значение исходной функции в найденной точке;
6. проверяет условие окончания;
7. при необходимости добавляет новую точку;
8. повторяет процесс.

При этом поиск не заменяется перебором функции на мелкой сетке.

Это важно, поскольку в задании отдельно указано, что использование мелкого разбиения вместо расчёта точек пересечения и ведения верхних и нижних вершин является ошибочным подходом.

Представленный алгоритм реализован для случая нахождения глобального минимума, но если мы хотим найти глобальный максимум достаточно умножить функцию $W(x)$ на коэффициент -1. таким образом, мы превратим задачу поиска максимума в задачу поиска минимума.

Поиск константы Липшица

По определению, константа Липшица должна удовлетворять неравенству для любых двух точек u, v на отрезке $[a, b]$.

$$|W(u) - W(v)| \leq L|u - v| \rightarrow \frac{|W(u) - W(v)|}{|u - v|} \leq L$$

т.е. верхняя граница наклона любой секущей (максимально возможная крутизна, с которой функция может расти на этом участке)

17. Теорема Лагранжа

Формула конечных приращений или теорема Лагранжа о среднем значении утверждает, что если функция непрерывна на отрезке и дифференцируема в интервале, то найдётся такая точка $c \in (a; b)$, что

$$\frac{f(b) - f(a)}{b - a} = f'(c).$$

Если функция дифференцируема, то по теореме Лагранжа о среднем значении, наклон секущей между любыми u и v равен производной в некоторой точке c между ними.

$$\frac{|W(u) - W(v)|}{|u - v|} = L = W'(c)$$

Так как точка c всегда лежит на нашем отрезке, наклон абсолютно любой секущей никогда не сможет превысить самое большое значение производной на этом отрезке.

Из чего следует, что минимально возможная константа L - максимум модуля ее производной.

$$L = \max |W'(x)|$$

То есть L должна быть достаточно большой, чтобы ограничивать скорость изменения функции на всём исследуемом отрезке. В теории работы именно такая константа используется при построении вспомогательных функций.

```

1. def find_L(func_str, a, b, n=20001):
2.     W = parse_function(func_str)
3.     xs = np.linspace(a, b, n)
4.     ys = np.array([W(x) for x in xs])
5.     slopes = np.abs(np.diff(ys) / np.diff(xs))
6.     return slopes.max()

```

Данная функция на заданном отрезке приближенно оценивает константу Липшица.

На вход функция принимает:

func_str - функция в виде строки, например "x + sin(pi*x)"

a, b - рассматриваемый отрезок

n - количество точек, в которых будет проверяться функция

В коде я по сути делаю то же самое, что описала выше, я решила посчитать все выражения, используя функцию

$$\frac{|W(u) - W(v)|}{|u - v|} = L = W'(c)$$

А затем взяла максимум.

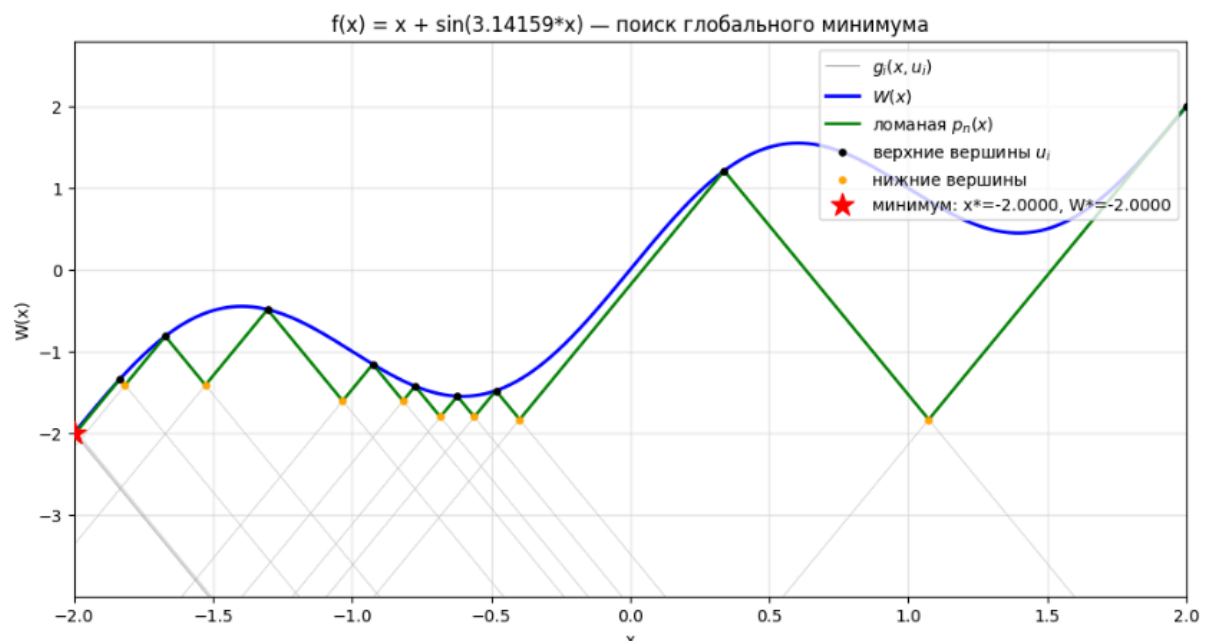
Результаты

В качестве тестовых функция я взяла предложенную в методичке функцию, а также функцию Растригина и функция Экли.

Для глобального минимума.

Функция из методички

Я решила с этой функцией попробовать 2 варианта, вставить заданную константу Липшица, а также приближенную.



ГЛОБАЛЬНЫЙ МИНИМУМ

Функция: $f(x) = x + \sin(3.14159 \cdot x)$

Отрезок: $[-2, 2]$

$\epsilon = 0.01$

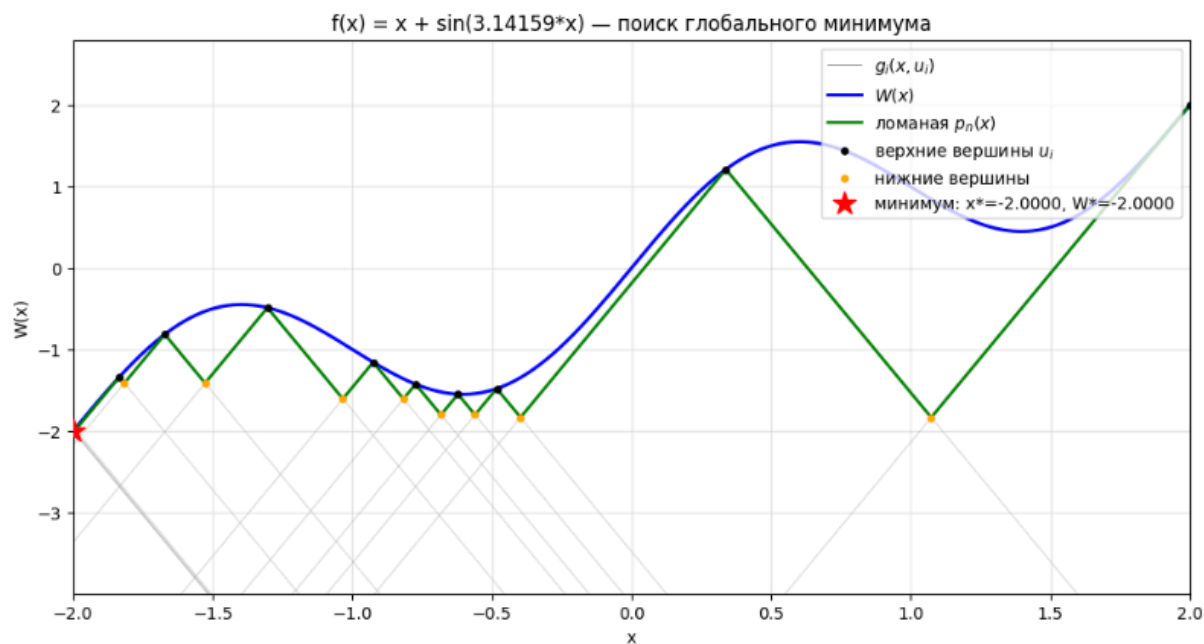
$L = 4.14159$

$x^* = -2.000000$

$W(x^*) = -1.999995$

Число итераций = 11

Время расчёта = 0.41 мс



ГЛОБАЛЬНЫЙ МИНИМУМ

Функция: $f(x) = x + \sin(3.14159 \cdot x)$

Отрезок: $[-2, 2]$

$\epsilon = 0.01$

$L = 4.141589793292017$

$x^* = -2.000000$

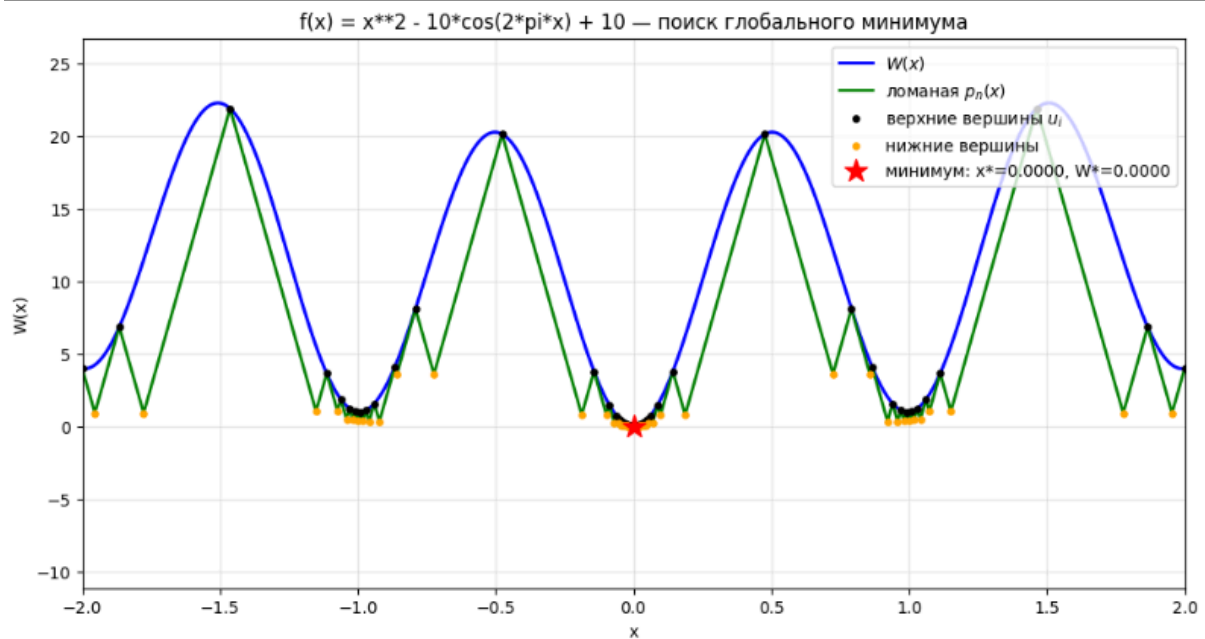
$W(x^*) = -1.999995$

Число итераций = 11

Время расчёта = 0.44 мс

Вывод: Результаты идентичны, но затраченное время во втором случае больше, из-за дополнительных расчетов.

Функция Растригина



ГЛОБАЛЬНЫЙ МИНИМУМ

Функция: $f(x) = x^2 - 10\cos(2\pi x) + 10$

Отрезок: $[-2, 2]$

$\epsilon = 0.01$

$L = 66.84$

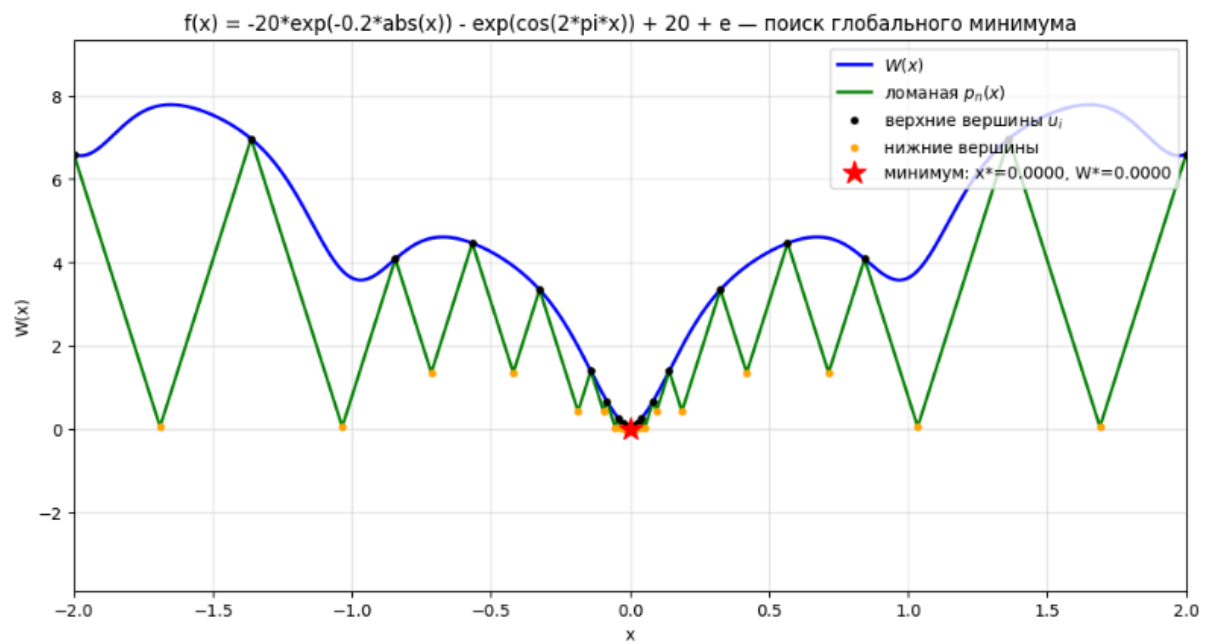
$x^* = 0.000000$

$W(x^*) = 0.000000$

Число итераций = 140

Время расчёта = 8.32 мс

Функция Экли



ГЛОБАЛЬНЫЙ МИНИМУМ

Функция: $f(x) = -20 \cdot \exp(-0.2 \cdot \text{abs}(x)) - \exp(\cos(2 \cdot \pi \cdot x)) + 20 + e$

Отрезок: $[-2, 2]$

$\epsilon = 0.01$

$L = 21.1$

$x^* = 0.000000$

$W(x^*) = 0.000000$

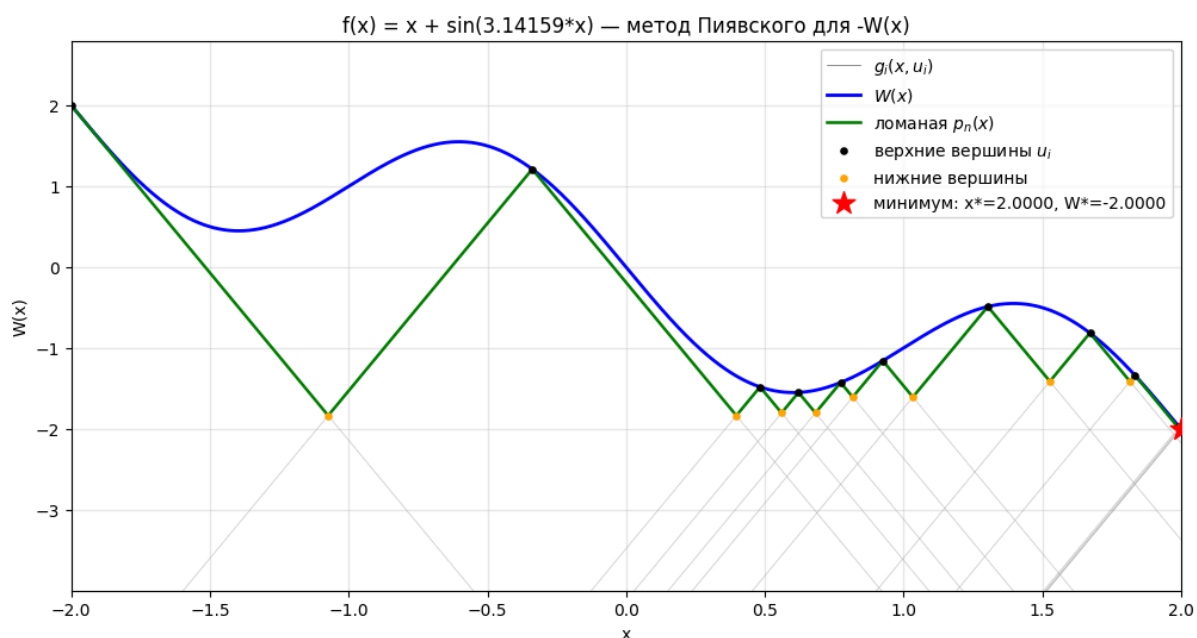
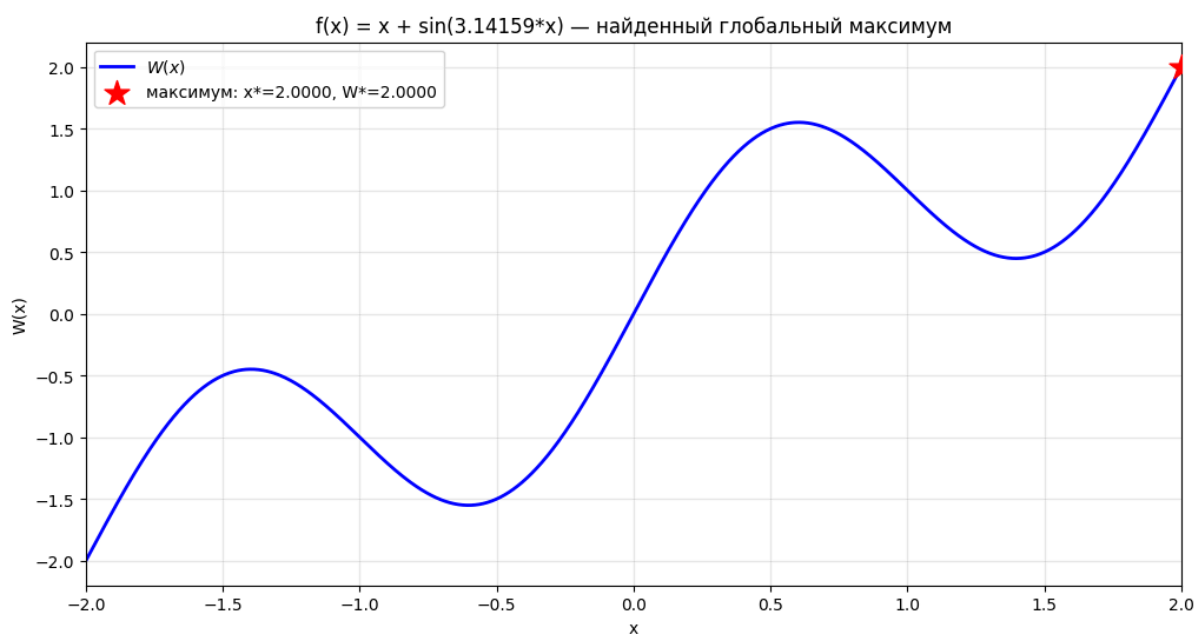
Число итераций = 32

Время расчёта = 1.57 мс

Для глобального максимума.

Так как при поиске глобального максимума мы умножаем функцию на -1, то будет некорректно отображать метод ломаных на исходной функции, поэтому было сделано 2 вывода: вывод исходной функции и вывод $-W(x)$ с отображением метода Пиявского.

Функция из методички



ГЛОБАЛЬНЫЙ МАКСИМУМ

Функция: $f(x) = x + \sin(3.14159 \cdot x)$

Отрезок: $[-2, 2]$

$\text{eps} = 0.01$

$L = 4.14159$

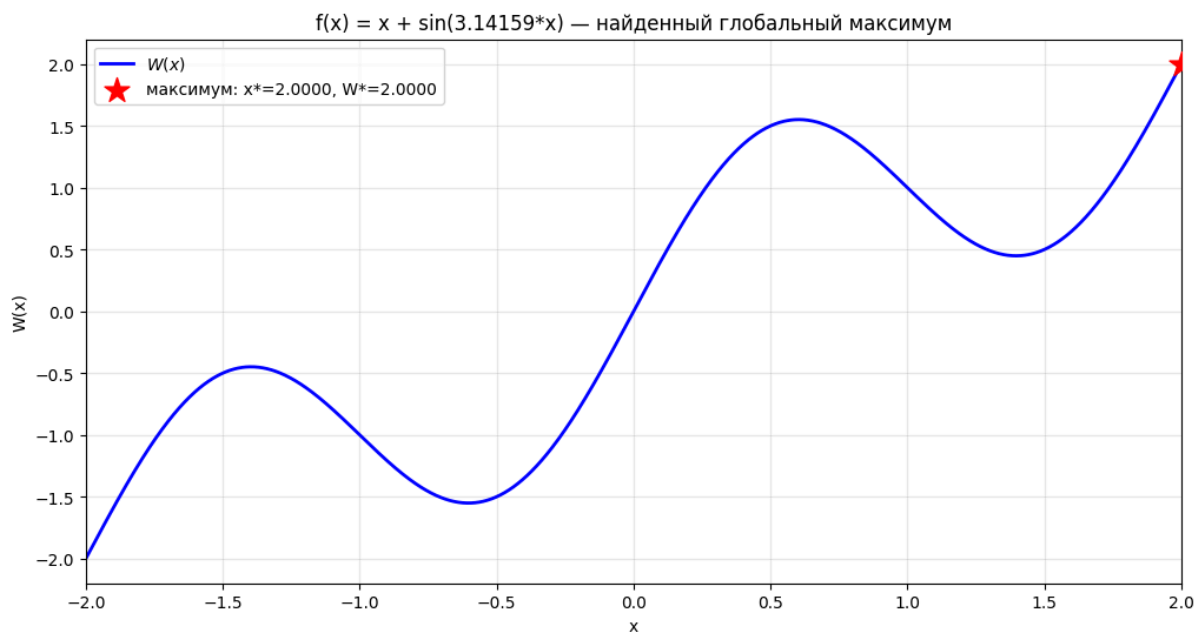
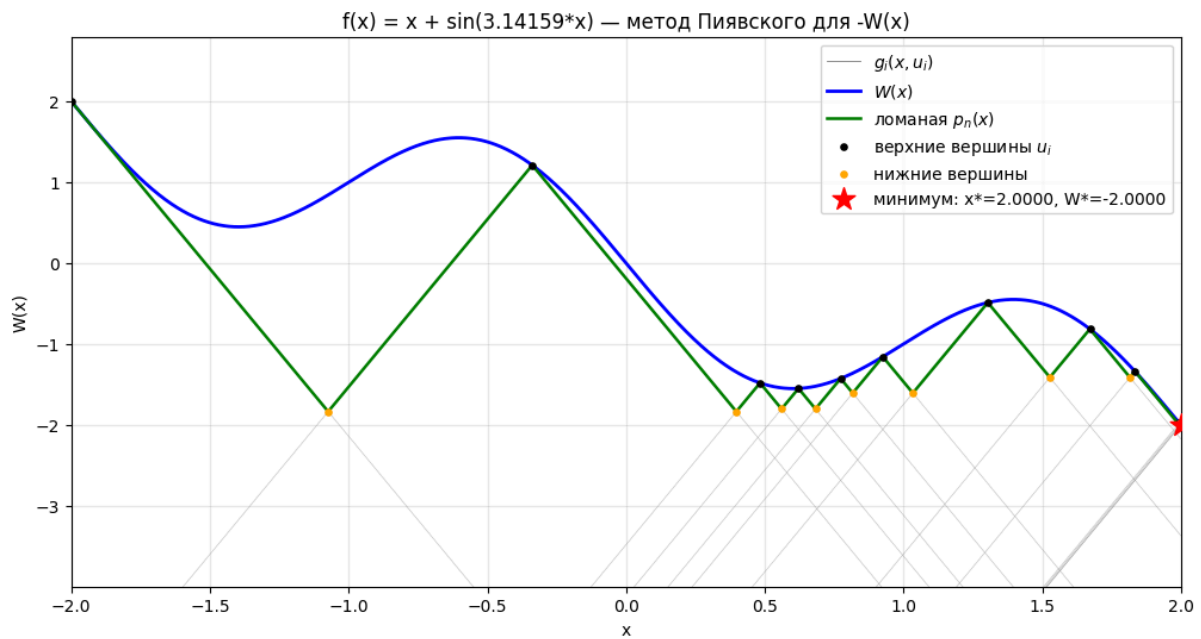
$x^* = 2.000000$

$W(x^*) = 1.999995$

Число итераций = 11

Время расчёта = 0.46 мс

При приближенном значении L :



ГЛОБАЛЬНЫЙ МАКСИМУМ

Функция: $f(x) = x + \sin(3.14159 \cdot x)$

Отрезок: $[-2, 2]$

$\text{eps} = 0.01$

$L = 4.141589793292017$

$x^* = 2.000000$

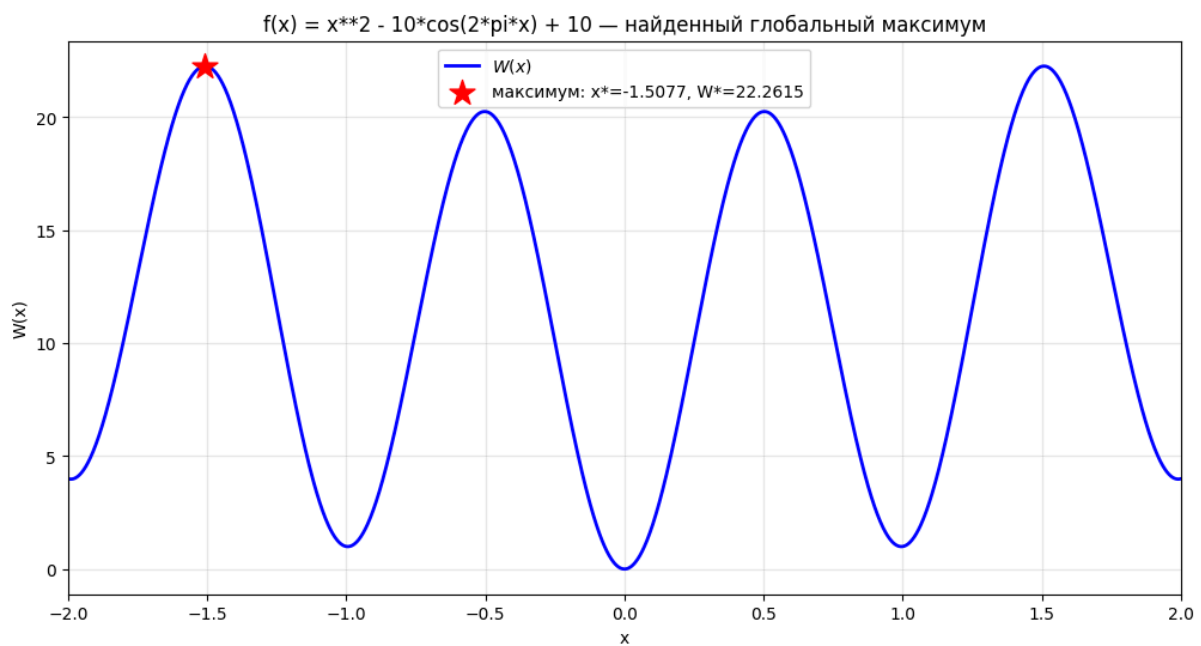
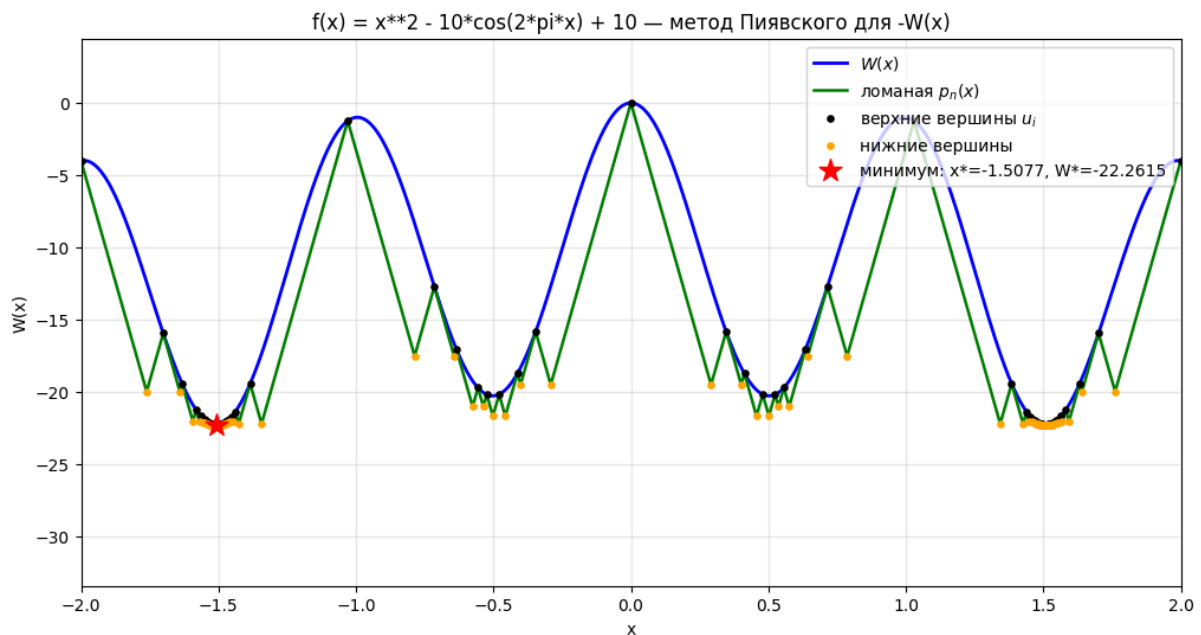
$W(x^*) = 1.999995$

Число итераций = 11

Время расчёта = 0.76 мс

Вывод: В данном случае получилось, что во втором случае у нас время затрачивается больше.

Функция Растригина



ГЛОБАЛЬНЫЙ МАКСИМУМ

Функция: $f(x) = x^2 - 10\cos(2\pi x) + 10$

Отрезок: $[-2, 2]$

$\epsilon = 0.01$

$L = 66.84$

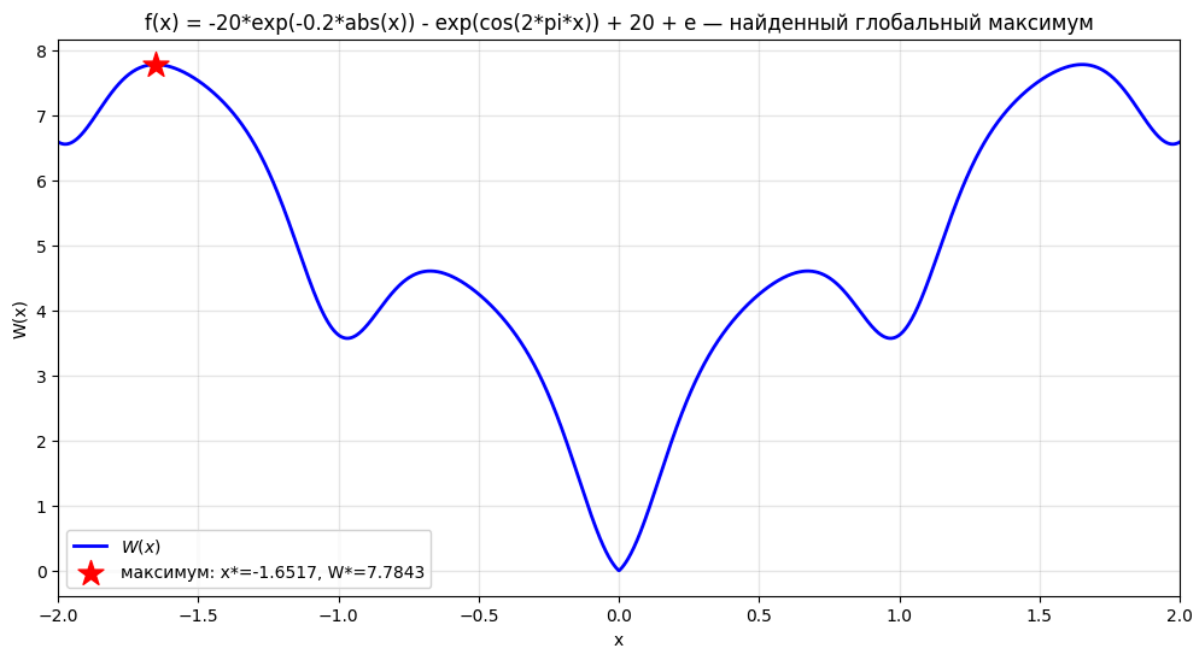
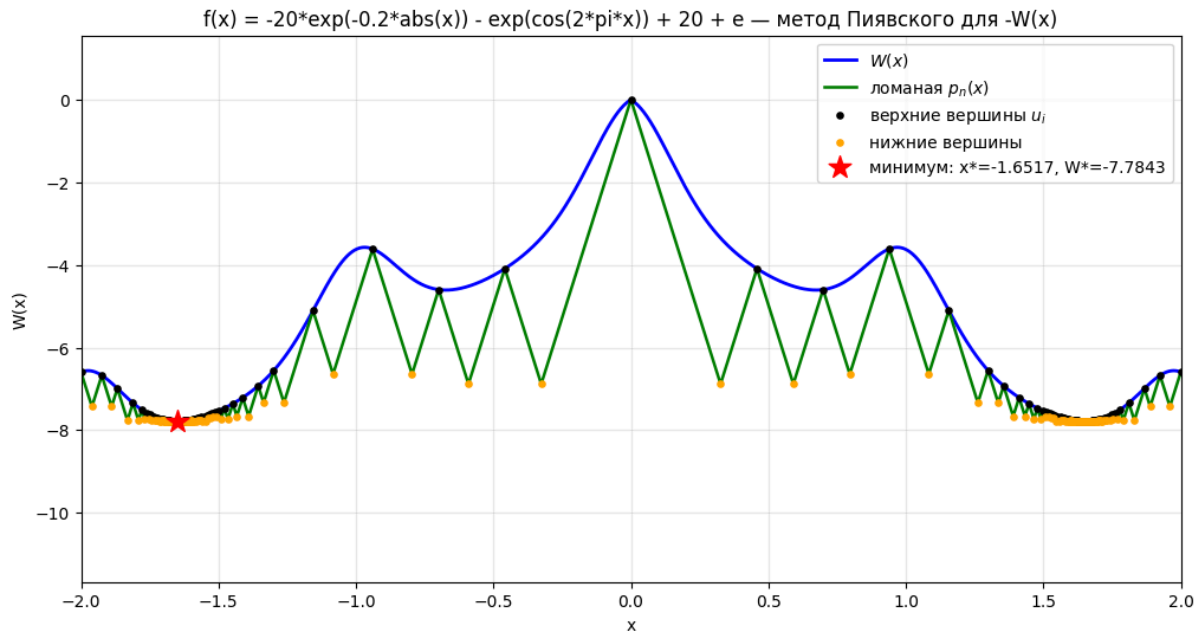
$x^* = -1.507704$

$W(x^*) = 22.261458$

Число итераций = 232

Время расчёта = 18.73 мс

Функция Экли



ГЛОБАЛЬНЫЙ МАКСИМУМ

Функция: $f(x) = -20 \cdot \exp(-0.2 \cdot \text{abs}(x)) - \exp(\cos(2 \cdot \pi \cdot x)) + 20 + e$

Отрезок: $[-2, 2]$

$\text{eps} = 0.01$

$L = 21.1$

$x^* = -1.651681$

$$W(x^*) = 7.784298$$

Число итераций = 304

Время расчёта = 35.98 мс

Вывод по лабораторной работе

В ходе лабораторной работы был реализован метод Пиявского(ломаных) для поиска глобального экстремума одномерной функции на заданном отрезке.

В ходе тестирования программа успешно выполняла поиск глобальных минимумов и максимумов исследуемых функций.

Для функции Растригина глобальный минимум был найден в точке $x = 0$, где значение функции равно нулю.

Для функции Экли глобальный минимум также был найден в точке $x = 0$ со значением функции, близким к нулю.

Количество итераций зависит от вида функции и направления оптимизации. Для некоторых функций поиск максимума потребовал больше итераций, чем поиск минимума. Это связано с формой исследуемой функции и особенностями построения миноранты для преобразованной функции $-W(x)$.

Во время работы метода новые точки выбираются не равномерно по всему отрезку, а в наиболее перспективных местах - в нижних вершинах текущей ломаной. Поэтому алгоритм последовательно уточняет область расположения глобального экстремума.

Ссылка на google_colab: [🔗 LP2_метОпт_Степанова](#)